

Performance for AI, AI for Performance

30 years of performance work, and the LLM era's cost and energy

Kingsum 周经森

Professor, Zhejiang University · Director, SPAIL Lab

Founder, Zhejiang Youtune Technology

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon

上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon

北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

■ Thirty years of making software run faster and cheaper on hardware



Contents

01 How expensive, how power-hungry

02 Performance for AI

03 Fundamentals

04 AI for Performance

05 Out of the data center

06 Five takeaways

01

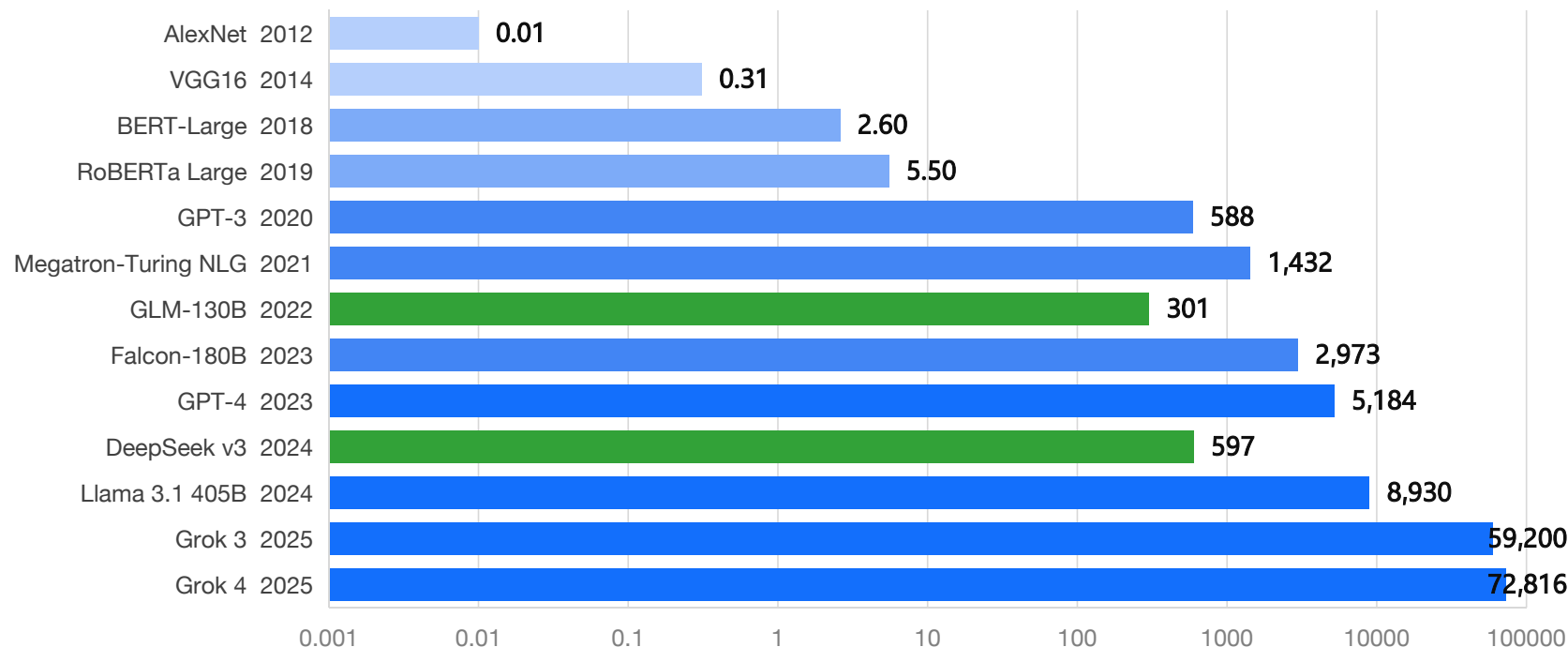
How expensive, how power-hungry

And why that is exactly where performance work pays

■ Training cost comes first: energy and carbon per run

AI Index 2026 estimates: **one training run per model; CO2e (Carbon dioxide equivalent) tons on a log scale**

Estimated training emissions (t CO2e, log scale; GPT-3 = 588 t)



FOR SCALE

One flight 0.99 t

World avg/person-year 5.51 t

US avg/person-year 18.08 t

One car, whole life 63 t

● Green = Chinese models

DeepSeek-V3 is larger than Llama 3.1 405B, yet its reported training emissions are **an order of magnitude lower**.

Source: Stanford HAI, AI Index Report 2026, Figure 1.4.3 | Common estimation method across the chart; one training run per model

Inference cost is recurring: every query adds to the bill

\$694K per day

ChatGPT inference operating cost

Feb 2023 third-party estimate: dense GPT-3 175B, ~29,000 A100s, 13M daily users; compute hardware only

0.36 cents per query

Cost of a single query

The same estimate stated differently; identical basis

\$21.2K per month

Small-business support desk

Not any company's real invoice: 360K queries/month, 1,800 input + 80 output tokens each, at 2023 list prices

High infrastructure and capital cost, plus scarce expertise, keeps large-model technology out of reach for most organizations.

— paraphrase of the abstract of "Performance Optimization in the LLM World", ICPE 2024 Companion

Price curve: Stanford HAI, AI Index Report 2025, Ch. 1 p. 5 | Speaker's own work: Chow et al., "Performance Optimization in the LLM World", ICPE'24 Companion | Cost figures: SemiAnalysis, "The Inference Cost of Search Disruption", 2023-02-09; Chen, Zaharia, Zou, "FrugalGPT", arXiv:2305.05176 §2 | the line above is a paraphrase, not a verbatim quotation

■ Why performance work scales with AI demand

Efficiency gains compound **at AI-infrastructure scale.** Scale effect

465 TWh / year

AI-dedicated data-center power in 2030 (projected)

IEA base case

4 TWh / year

Added load if all web search became AI answers

IEA comparison estimate

46.5 TWh / year

Energy saved if the same workload uses 10% less energy

10% of the projected 465 TWh base case

At the same workload, using 10% less energy saves **46.5 TWh per year**—more than **10** times the projected 4 TWh search load.

Source: IEA, Key Questions on Energy and AI, 2026-04, pp. 28, 30—31 and Figure 2.2

02

Performance for AI: an inference-led tour

Model, runtime, and system — from GPU kernels to domestic platforms

What one inference request has to go through

Performance diagnosis spans three interacting layers

Layer 1 The model

Quantization · pruning · distillation · operator fusion

changes **WHAT** is computed

Layer 2 Runtime

KV cache · caching · batching · co-location · scaling

changes **HOW** it is scheduled

Layer 3 System & HW

Heterogeneous
scheduling · compilers · microarchitecture · SMT

changes **WHERE** it runs

The limiting factor can move across layers as the **workload and hardware change**.

Source: Chen et al., arXiv:2509.20979v2, first sentence of §1

DeepSeek-V3: three layers that reduce inference cost

Co-design across algorithms, frameworks, and hardware

Layer 1 The model

Sparse MoE: 37B of 671B parameters active per token
MLA compresses the KV state kept during generation

Layer 2 Runtime

MTP (Multi-token prediction) can drive speculative decoding
85–90% second-token acceptance; 1.8× tokens/s

Layer 3 System & HW

Optimized kernels and model–hardware co-design
Efficient inference still requires a large deployment unit

INFERENCE PAYOFF

37B / 671B

parameters active per token

1.8×

tokens/s with MTP decoding

Smaller KV state

Multi-Head Latent Attention (MLA)
caches compressed latent vectors;
less memory and bandwidth per
generated token

The inference gains come from **sparse activation, KV compression, and speculative decoding**.

Sources: DeepSeek-V3 Technical Report, arXiv:2412.19437, Abstract, §2.1.1 and §5.4.3; ISCA 2025 industry track, arXiv:2505.09343 | Training-cost figures are omitted from this inference section

Quantization: what it saves, and why deployment is hard

Lower-precision numbers reduce model memory and arithmetic cost; deployment requires a policy that transfers across architectures.

APPROACH 1 — ARCHITECTURE-SPECIFIC

Q-Diffusion: hand-coded rules

Rule is written for one UNet skip-connection pattern

A different architecture exposes different graph boundaries and activation distributions

Porting it requires a new hand-written rule

APPROACH 2 — RUNTIME-DEPENDENT

PTQ4DiT: runtime activation data

Activation statistics change with each diffusion time step

The static compiler runs before those values exist

Runtime-only evidence cannot directly drive a compile-time policy

WHY THIS BLOCKS AUTOMATION: "compiler gap"

Both approaches can work on one model, but architecture-specific rules and runtime-only data prevent portable automation.

The deployment target is **an automatic policy that generalizes across models.**

Source: SegQuant, CVPR 2026, Abstract and §1 | Scope: diffusion models; this claim is not generalized to all post-training quantization

■ Case: SegQuant (CVPR 2026)

Quantization policy derived entirely from the static graph — no hand-written rules, no runtime values

W8A8

Backed by quality numbers

1.919× **Model size**
3.88 → 2.02 GiB

1.102× **Per-step latency**
317.2 → 287.9 ms

1.114× **Peak GPU memory**
18.63 → 16.72 GiB

W4A4

No quality table given for it

2.738× **Model size**
3.88 → 1.42 GiB

1.490× **Per-step latency**
317.2 → 212.9 ms

SegLinear derives the boundaries; **DualScale** uses the native dual path — no custom operators.

Sources: SegQuant, CVPR 2026, §4.2.2, §4.3.2, §5.1 and Figure 10; code at github.com/OptiSys-ZJU/segquant | The two configurations are different; their numbers must not be merged into one claim

The cheapest move here: compute only part of the model

Quantization makes each pass cheaper; a cheaper move is to keep most parameters out of the pass

Model	Parameters	Activated per token	Training energy	CO2e
GPT-3	175B	100%	1,287 MWh	552 t
GShard-600B	600B	0.25%	24 MWh	4.3 t
Switch Transformer	1.5T	0.10%	179 MWh	59 t

GShard: **4.3 t CO2e** vs GPT-3: **$552 \div 4.3 \approx 128$, so about 1/128.**

What pays off in this layer is not computing faster, it is **computing less each time.**

My judgment

All three rows come from one paper, one table, one measurement method (Patterson et al., arXiv:2104.10350 Table 4) | Three Chinese teams converged on the same road: arXiv:2505.09343, Kimi Team arXiv:2507.20534, Qwen Team arXiv:2505.09388

A problem map for LLM-service latency—not one finished solution

Audience takeaway: telemetry matters only when it leads to an explicit resource decision

1 OBSERVE

Tail-latency prediction

dual-stream decoupling + gradient modulation

current snapshot → P95 tail latency

arXiv:2508.01635

2 ESTIMATE

MSGAF

multi-scale graph fusion

resource quota → latency estimate

arXiv:2604.16409

3 ACT

Morphis

time-varying call graph

end-to-end target → per-service resources

arXiv:2602.01044

Use this decomposition: observe tail latency, estimate quota effects, then allocate resources. The papers remain separate.

This is a **problem map, not an integrated pipeline.**

Sources: arXiv:2508.01635v2, arXiv:2604.16409v1, arXiv:2602.01044v2; DOI 10.3389/fcomp.2026.1783945 §6.4 | The three problems are shown side by side; this is not one connected pipeline

■ ■ Three production problems

What I see in LLM systems: shared accelerators, growing KV caches, and traffic-dependent model choices

1

Performance prediction and scheduling for co-located LLM services

Predict end-to-end latency under a quota, then schedule co-located model services on a shared accelerator pool

2

KV-cache optimization

The fastest-growing memory item in long-context inference, and a main source of the memory-access bottleneck

Runtime &
scheduling

3

Traffic-aware quantization

Tie the quantization configuration to real production traffic instead of taking one static policy from an offline calibration set

Model
deployment

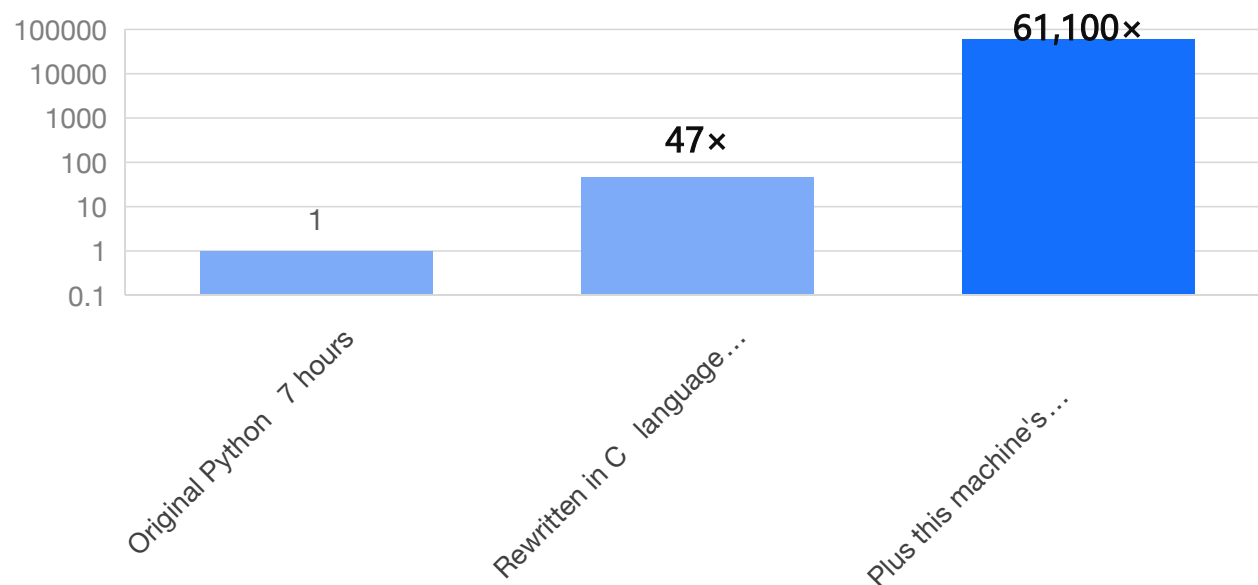
The common thread: **start from production traffic and resource constraints.**

Three directions: co-location scheduling, KV-cache efficiency, and traffic-aware quantization

Hardware-aware implementation can dominate language choice

CPU case: one 4096×4096 matrix multiply, one 18-core machine, three levels of implementation

Speedup vs. original Python (log scale)



WHAT “HARDWARE FIT” MEANS HERE

18 CPU cores · memory hierarchy

Vectorization · AVX instructions

FIXED MEASUREMENT BASIS

Same 4096-order matrix multiply on the same 18-core Intel CPU; metric = runtime and speedup

Changing Python to C yields **47x**; exploiting CPU parallelism and data paths yields another **1,300x**.

Source: Leiserson et al., Science 368:eaam9744, 2020, Software section and Table 1

Case: FlashAttention-2 on A100 GPUs

Three code changes map directly to A100 execution and memory behavior

1.7–3.0×

**faster than FlashAttention in
attention benchmarks**

50–73%

**of theoretical peak FLOPs/s on
A100**

1.3×

**up to end-to-end GPT training
speedup**

One paper, one GPU family: A100; kernel, peak-utilization, and end-to-end results use separate metrics

OPTIMIZATION → GPU EFFECT

Fewer non-matmul FLOPs → more time in tensor-core math; one head across thread blocks → more parallel work; less warp communication → fewer shared-memory reads and writes.

Exact attention is unchanged; **the work partitioning** is redesigned for **A100 execution and memory hierarchy**.

Source: Tri Dao, FlashAttention-2, ICLR 2024, arXiv:2307.08691, Abstract and §4 | Attention-benchmark and end-to-end speedups are different measurements and are not merged

03

Fundamentals

■ ■ Five measurement checks to run before optimizing

1

Measure the tool's own overhead

One Java profiler cost 20% of throughput and changed the JVM's own behaviour

2

Lock and record CPU frequency

The tool reported 45% utilization while the CPU core ran at 1.6 GHz, not the normal 2.93 GHz

3

Verify what each metric actually computes

One reports a 10-second mean, another the instant value at second 10, a third the value at second 5 of the window

4

Inspect detailed data, not only averages

The whole-run average throughput gave no hint of this drop; only the detailed data shows it

5

Sweep the operating range before concluding

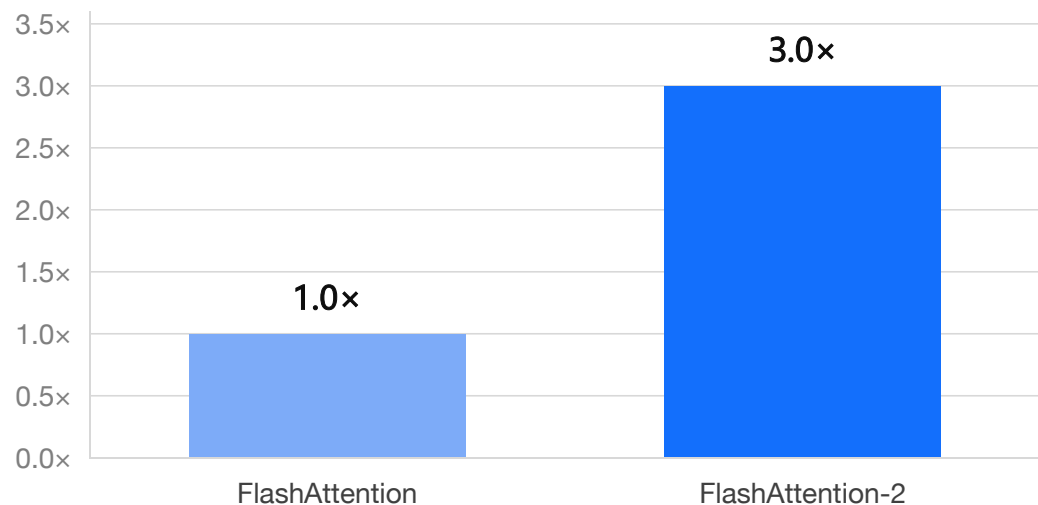
At high concurrency B was 24% faster than A; with fewer users A was 2% faster than B

All five come from the speaker's own published work: PNSQC 2011 §2, §3.2, §4; "Java Performance Mysteries", ITM 2016, §3.1, §3.5; PNSQC 2016 §3.2 | Corroboration: Mytkowicz et al., ASPLOS 2009 — of the 133 papers with experimental results they surveyed, not one adequately accounted for measurement bias

■ ■ Why a 3.0× kernel speedup becomes 1.3× end to end

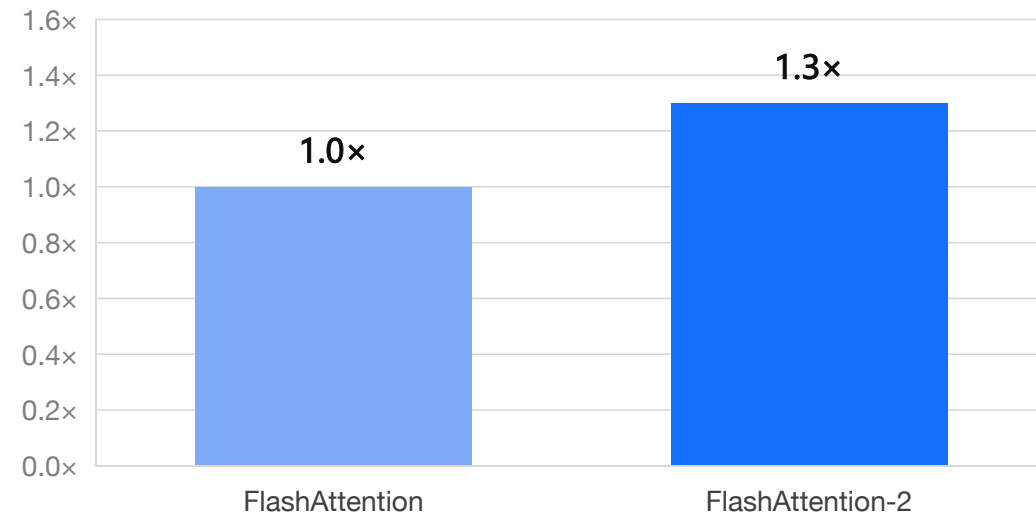
Attention is only part of GPT training; other kernels, communication, and training overhead remain unchanged.

PART OF THE WORKLOAD — attention microbenchmark



Attention-only result **up to 3.0x**

WHOLE WORKLOAD — end-to-end GPT training



Whole-training result **up to 1.3x**

The relationship: **speeding up one component is diluted by unchanged work elsewhere.**

Both are reported maxima on A100; they use different scopes and cannot be divided or merged | Source: Tri Dao, FlashAttention-2, ICLR 2024, arXiv:2307.08691, §4

Two models, different metric, the conclusion flips

Training happens once; inference happens every day

Training emissions (t CO2e)

Llama 3.1 405B

8,930

DeepSeek-V3

597

DeepSeek ~15× better

Energy per medium-length query (Wh)

Llama 3.1 405B

9.00

DeepSeek-V3

15.86

DeepSeek 1.76× worse

Emissions per medium-length query (g)

Llama 3.1 405B

2.70

DeepSeek-V3

9.51

DeepSeek 3.5× worse

15.86 Wh is the highest among the models measured for 2024 in that report

Trap 3, live: **training and inference are two separate books.**

Source: Stanford HAI, AI Index Report 2026, Ch. 1, the training-emissions and inference-energy sections | In the same figure, self-hosted and Azure-hosted DeepSeek-V3 inference emissions differ by an order of magnitude; that gap is driven mainly by grid carbon intensity, not model efficiency

How to read and report inference benchmarks

Purpose: disciplined comparison—not a new benchmark and not a claim that MLPerf is inadequate

EXAMPLE: WHAT MLPERF DIVISIONS CONTROL

Closed and open divisions

The closed division keeps the reference model and quality target on a common basis; the open division permits model changes and reports them separately.

CHECKLIST FOR ANY REPORTED RESULT

Keep one basis for every comparison

Model, precision, batch size, latency target, quality target, hardware count, and power boundary determine what an inference result means.

A COMPLEMENTARY NORMALIZATION

Resource-use effectiveness

Effectiveness = resource used ÷ work completed

lower is better

Speedup between configurations = ratio of two effectiveness values

higher is better

Resources: accelerators, CPU, memory, storage, network, energy

Takeaway: compare only results sharing the same **workload, quality target, operating point, and reporting rules**.

Sources: MLCommons, MLPerf Inference: Datacenter—Divisions, Scenarios and Metrics | Resource-use effectiveness is defined on SREcon21 official slides 10 and 11

Reproducibility

A performance gain that cannot be reproduced cannot support an engineering decision

APPROACH 1 The two-engineer parable, made a rule

- 1 Two people, different languages and packages, each write a script over the same raw data
- 2 Compare the two results item by item, down to attribute names and order
- 3 Already implemented in our tooling, and open to third-party scripts

APPROACH 2 Automatic sanity rules

- × CPI cannot be below 0.25
- × CPU utilization cannot exceed 100%
- × Missing values cannot outnumber timestamps, or the attribute is just noise

Performance engineering is experimental science — this is not a metaphor ASPLOS 2009 borrowed "measurement bias" from the natural and social sciences

My first requirement for students: honesty, passion, curiosity, and **real reproducible work**.

My judgment

Sources: Chen, Jain, Chow, Guirguis, Wu, "Brewing Analytics Quality for Cloud Performance", PNSQC 2015 §2.1, §4.6; Georges, Buytaert, Eeckhout, OOPSLA 2007 abstract; Mytkowicz et al., ASPLOS 2009

■ ■ Turning those rules into one checklist

Answer these five questions before you state any performance conclusion

1

Did my measuring tool itself change what is measured?

2

Which variables did I control? Could the uncontrolled ones explain the difference I see?

3

Is this metric really measuring what I think it measures?

4

If someone else repeats it, does the conclusion still hold?

5

If the result is "no gain", do I doubt the method first, or the approach?

Number five is the one that matters: **doubt the measurement before the approach**

The five map to: "Java Performance Mysteries" 2016 §3.5 | PNSQC 2011 §2 | ICPE 2024 tutorial slide 34 | last slide of the SREcon21 deck | PNSQC 2015 §2.1 and §4.6

04

AI for Performance

I have been on this road for twenty years — it just wasn't called AI

■ Twenty years of this, back when it wasn't called AI

2000	●	US patent US6684252B1	Method and system for predicting computer-server performance; my only solely-authored patent, assigned to Intel
2006	●	IISWC paper	A multilayer perceptron for non-linear workload characterization — twenty years ago
2014	●	US patent US10089207B2	Identifying software phases with machine learning; European family member EP2960797B1 still in force
2015	●	US patents US9954744B2, US10452443B2	Estimating application performance change; dynamic adjustment on multiprocessor and multicore systems
2017	●	Two PCT international applications	Processing performance data for machine learning; latency-anomaly detection in Java. Applicant Alibaba; never entered national phase
2022	●	ICPP paper	Microarchitecture profiling of large-scale jobs: dataset and analysis

This road went nowhere before, blocked on data and models; **today we have both.**

My judgment

Sources: IISWC 2006 DOI 10.1109/IISWC.2006.302739; USPTO granted-patent front pages; WIPO PATENTSCOPE | The two 2017 filings have international publication only and never entered national phase

■ ■ Today: six concrete directions in AI for Performance

Six of the workshop's ten call-for-papers topics belong to "AI for performance engineering"

1 Data-driven model identification for performance evaluation

2 White-box performance modelling

3 Datasets and benchmarks for training and validating AI performance models **connector**

4 Explainability and robustness of AI systems used in performance engineering

5 AI models for anomaly detection, classification, root-cause analysis and repair

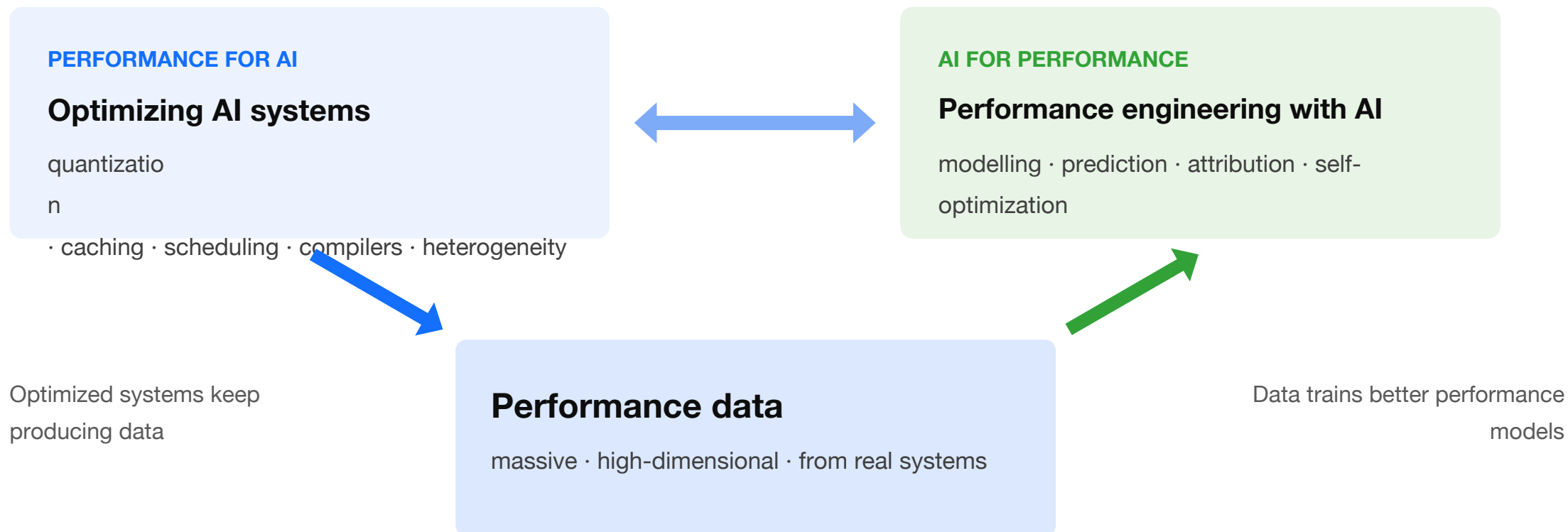
6 Automating performance tasks: autoscaling, self-optimization

Not speculative: **the cache decisions and tail-latency prediction** came from this side.

Source: AIPerLLM 2026 official call for papers (ICPE 2026) | The highlighted item is topic 7, "datasets and benchmarks for training and validating AI performance models", which is exactly what connects the two directions

■ ■ Not two things — one loop

Not two parallel activities, but one loop that turns



Of the ten CFP topics the first four are Performance for AI and the last six are AI for Performance; topic 7 — datasets and benchmarks for AI performance models — connects them.

Source: AIPerfLLM 2026 official call for papers (ICPE 2026)

Where I put that judgment: AIPerfLLM

Four consecutive editions of the LLM-world performance workshop at ACM/SPEC ICPE

1st edition

May 2024 London

3rd edition

May 2025 Toronto

4th edition

5 May 2026 just held

TEN CO-ORGANIZERS

AMD · Intel · York University

IMT School for Advanced Studies Lucca · Gran Sasso

Science Institute

University of Waterloo

TEN CFP TOPICS

The first four are Performance for AI, the last six are AI for Performance

Topic 7, datasets and benchmarks for training and validating AI performance models, is exactly what connects them

What this field lacks most is **public datasets and benchmarks**.

Not self-promotion — an invitation

Source: AIPerfLLM 2026 workshop site, organizer list and call for papers (ICPE 2026) | Four consecutive editions; only the three with time and place verifiable on the site are marked above

05

Out of the data center

What comes next is optimizing machines that move

■ ■ Next: performance work leaves the data center

Thirty years were mostly about data centers; next comes machines that move

	what the last thirty years optimized	what comes next
	The data-center era	The embodied-AI era
What is optimized	Workloads on servers	Physical agents that move, see and decide
Main constraints	Throughput, latency, power	Plus real time, limited comms, battery, safety
System levels	Cloud	Cloud — edge — device, three tiers
Cost of failure	Requests get slower	The arm hits something; the inspection misses

The IEA notes that AI converging with robotics, autonomous driving, industrial control and edge devices will push electricity demand outside the data center; embodied and real-time systems need low-latency inference, continuous sensing and on-device processing.

The cost of failure moves from "200 ms slower" to **"something breaks in the physical world"**.

My judgment

Source: IEA, "Key Questions on Energy and AI", 2026-04, p. 32 | Deployment settings: robotic-arm assembly and inspection, quadruped inspection robots; Zhejiang Yutiao Technology, founded May 2025, is the vehicle for taking this to market

Five takeaways

All five are directly actionable

1

Look at all three layers first

The bottleneck is usually not the layer you watch

2

Generality is performance

An optimization welded to one architecture expires with that architecture

3

Doubt the measurement first

But once it is clean, accept "it simply does not help"

4

Build your own minimal performance validation set

No stable reproduction, no optimization

5

Keep the optimization data

It is the fuel for training your next performance model

Behind number four: two contracted projects on the minimal performance validation set | Behind number five: ICPP 2022, DOI 10.1145/3545008.3545026 — the "dataset" in that title is itself the contribution

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

Make AI run, and make performance smarter

AI for Performance, Performance for AI

kingsum.chow@zju.edu.cn | kingsum.github.io

SPAIL Lab zju-spail.github.io

AiCon

全球人工智能开发与应用大会

SPAIL Lab recruits postdocs and graduate students in performance analysis and optimization